

บทที่ 4

การเขียนดีไวซ์ไดรเวอร์

4.1 การเขียนไดรเวอร์สำหรับลินุกซ์

อุปกรณ์แต่ละชนิดย่อมมีลักษณะการทำงานที่แตกต่างกันออกไป ซึ่งโอเอส (OS) มีหน้าที่ควบคุมและติดต่อกับอุปกรณ์ต่าง ๆ ทุกประเภท ดังนั้นโอเอสจึงจำเป็นต้องรู้ลักษณะการทำงานของอุปกรณ์ทุกประเภท เพื่อจะได้สามารถควบคุมการทำงานของอุปกรณ์แต่ละประเภทได้ถูกต้อง แต่หากเรานำเอาโปรแกรมที่เป็นส่วนควบคุมอุปกรณ์ทั้งหมดมาใส่รวมไว้ในตัวโอเอส จะทำให้โอเอสมีขนาดใหญ่มาก ผู้ออกแบบโอเอสจึงมีความคิดที่จะแยกเอาส่วนที่ควบคุมอุปกรณ์ต่าง ๆ ออกจากตัวโอเอส ซึ่งโปรแกรมที่แยกออกมานี้ เรียกว่า ดีไวซ์ไดรเวอร์ (Device Driver) เมื่อโอเอสต้องการที่จะติดต่อกับอุปกรณ์ใด โอเอสจะทำการติดต่อผ่านทางดีไวซ์ไดรเวอร์ชนิดนั้นแล้วดีไวซ์ไดรเวอร์จะทำการติดต่อกับอุปกรณ์จริงเอง

ความหมายของดีไวซ์ไดรเวอร์

ดีไวซ์ไดรเวอร์เป็นโปรแกรมพิเศษที่ถูกโหลดเมื่อระบบทำงาน มีหน้าที่ในการเชื่อมต่อการทำงานระหว่างแอปพลิเคชัน (Application) ของผู้ใช้งานกับอุปกรณ์ฮาร์ดแวร์ (Hardware) ซึ่งควรออกแบบให้มีความยืดหยุ่นในการใช้งาน ไม่จำกัดการใช้งานฮาร์ดแวร์ โดยให้ตัวแอปพลิเคชันเป็นตัวเลือกการทำงานเอง ส่วนดีไวซ์ไดรเวอร์ทำหน้าที่เพียงจัดการทรัพยากร (Resource) ภายในของคอมพิวเตอร์ให้เหมาะสมแก่การใช้งาน

ลินุกซ์จะแยกประเภทของดีไวซ์ (Device) ออกเป็นหลัก ๆ 3 ชนิด แต่ละมอดูลปกติจะสร้างหนึ่งในสามแบบนี้คือ char module, block module และ network module

- **คาแรกเตอร์ดีไวซ์ (Character Devices)**

คาแรกเตอร์ดีไวซ์เป็นแบบหนึ่งที่สามารถเข้าถึงได้โดยกระแสข้อมูลแบบไบนารี (stream of bytes) เหมือนกับไฟล์ ดังนั้นการสร้างฟังก์ชันการทำงานจึงเหมือนกับฟังก์ชันของไฟล์ เช่นอย่างน้อยต้องสร้างมีฟังก์ชัน open(), close(), write() และ read() ที่เป็นซิสเต็มคอลล์ (system calls) ยกตัวอย่างคาแรกเตอร์ดีไวซ์เช่น ซีเรียลพอร์ต (serial ports)

- **บล็อกดีไวซ์ (Block Devices)**

บล็อกดีไวซ์คล้ายกับคาแรกเตอร์ดีไวซ์ แต่บล็อกดีไวซ์จะถูกเข้าถึงได้ด้วยไฟล์ซิสเต็ม (file system nodes) ในไดเรกทอรี /dev บล็อกดีไวซ์สามารถจัดการได้ด้วยไอโอโอเปอร์เรชัน (I/O Operations) ที่เคลื่อนย้ายข้อมูล (transfer) เป็นบล็อก ๆ เท่านั้น ความแตกต่างกันระหว่างคาแรกเตอร์ดีไวซ์และบล็อกดีไวซ์คือการจัดการกับการเคลื่อนย้ายข้อมูล

● เน็ตเวิร์คดีไวซ์ (Network Devices)

อุปกรณ์ที่ทำหน้าที่รับหรือส่งแพ็กเก็ตข้อมูล ซึ่งทำงานร่วมกับบีเอสดีซ็อกเก็ต (BSD Socket) ผ่านข้อมูลโดยใช้ซ็อกเก็ตบัฟเฟอร์ (Socket Buffer) เพื่อประมวลผลตามมาตรฐานของโพรโทคอล (Protocol) ชั้นต่าง ๆ จนถึงการติดต่อกับอุปกรณ์ฮาร์ดแวร์จริง

การสร้างและรันงานมอดูล (Building and Running Modules)

การเขียนดีไวซ์ไดรเวอร์ คือ การเขียนโปรแกรมเพื่อรันในส่วนของลินุกซ์เคอร์เนล ซึ่งไดรเวอร์จะรันบนเคอร์เนลโหมด (Kernel mode) โดยไดรเวอร์บนลินุกซ์จะถูกแบ่งออกเป็น 2 แบบ คือแบบบิวท์อิน (Build-in) และแบบมอดูล (Module) ซึ่งจะต่างกันที่ช่วงเวลาที่ใช้เรียกการทำงาน

ลินุกซ์มอดูลเป็นเซตของฟังก์ชันและชนิดของข้อมูล ซึ่งสามารถจะคอมไพล์ (Compile) แยกจากตัวเคอร์เนลได้ และจะถูกเรียกใช้งานหลังจากที่บูตเคอร์เนลแล้ว มอดูลจะใช้ในการอิมพลีเมนต์ (Implement) ดีไวซ์ไดรเวอร์ ซึ่งจะใช้ติดต่อกับอุปกรณ์เฉพาะของไดรเวอร์นั้นเท่านั้น

ขั้นตอนในการเขียนโปรแกรมดีไวซ์ไดรเวอร์

1. เรียนรู้การทำงานภายในของโอเอสที่ใช้งาน เช่น วินโดวส์ หรือ ลินุกซ์ เป็นต้น
2. เรียนรู้ว่าจะเขียนดีไวซ์ไดรเวอร์อย่างไรบนโอเอสที่ใช้งานอยู่เช่น DDK, DDI หรือ DKI เป็นต้น
3. หา Tool มาใช้สำหรับการพัฒนาและการดีบักในเคอร์เนลโหมด
4. ทำการเขียนดีไวซ์ไดรเวอร์บนเคอร์เนลโหมด ซึ่งเป็นไดรเวอร์พื้นฐานของฮาร์ดแวร์ที่เป็นส่วนติดต่อกับอินพุตและเอาต์พุต
5. ทำการเขียนแอปพลิเคชันในยูสเซอร์โหมด (User mode) ซึ่งสามารถจะใช้ฮาร์ดแวร์ผ่านทางดีไวซ์ไดรเวอร์ที่เขียนอยู่ในเคอร์เนลโหมดได้
6. ทำขั้นตอนที่ 1-4 ใหม่ สำหรับเมื่อมีการเปลี่ยนเป็นโอเอสตัวใหม่ ซึ่งควรที่จะสามารถรันโค้ดในข้อ 5 ได้

4.2 โปรแกรมมอดูล

แบ่งเป็น 2 ส่วนหลัก ๆ คือ

1. `module_init(function)` : จะถูกเรียกใช้งานเมื่อมีการติดตั้งมอดูลเข้าไปในเคอร์เนล ใช้ในการติดตั้งค่าหรือเตรียมพร้อมการทำงานของมอดูลนั้น โดยฟังก์ชันนี้จะไปเรียก *function* มาทำงาน โดยเราจะต้องไปเขียนฟังก์ชันที่ใช้ในตอนเริ่มต้นโปรแกรม อย่างเช่น

```
static int hello_init(void)
{
    printk(KERN_ALERT "Hello, world\n");
    return 0;
}
```

แล้วก็มีประกาศ `module_init(hello_init);` ไว้ด้านล่าง

การเรียกใช้งาน : `root# insmod ./hello.ko` (สมมุติว่าหลังจากคอมไพล์แล้วได้ `hello.ko`)

2. `module_exit(function)` : จะถูกเรียกใช้งานเมื่อต้องการกำจัดมอดูลออกจากเคอร์เนลของระบบ ใช้ในการจัดการกำจัดสิ่งที่เราได้สร้างขึ้นมาจากโปรแกรมของเราให้ระบบมีสภาพเหมือนก่อนติดตั้งมอดูล โดยฟังก์ชันนี้จะไปเรียก `function` มาทำงาน โดยเราจะต้องไปเขียนฟังก์ชันที่ใช้ในตอนกำจัด อย่างเช่น

```
static void hello_exit(void)
{
    printk(KERN_ALERT "Goodbye, cruel world\n");
}
```

แล้วก็มีประกาศ `module_exit(hello_exit);` ไว้ด้านล่าง

การเรียกใช้งาน : `root# rmmod hello` (สมมุติว่าเราริเจิสเตอร์มอดูลชื่อว่า `hello`)

ที่กล่าวมาในการเขียนโปรแกรมดีไวซ์ใดเวอร์เราต้องมีการประกาศอินคลูด (`include`) เฮดเดอร์ไฟล์ที่จะเป็นด้วยซึ่งไฟล์ที่จำเป็นต้องอินคลูดมีดังนี้

```
#include <linux/init.h>
#include <linux/module.h>
```

4.3 การคอมไพล์

จะใช้เมกไฟล์ (`Makefile`) โดยการเขียนเมกไฟล์เบื้องต้นมีส่วนที่สำคัญดังนี้

```
obj-m := hello.o
```

เป็นการบอกว่ามีหนึ่งมอดูลที่จะสร้าง โดยสร้างจากออบเจ็กต์ไฟล์ (`object file`) ชื่อ `hello.o` ผลลัพธ์ที่ได้จะเป็น `hello.ko` หลังจากทีบิลด์ (`built`) จากออบเจ็กต์ไฟล์

แต่ถ้าเราต้องการมอดูลจากหลายไฟล์ เราสามารถทำได้ดังตัวอย่างด้านล่างนี้

```
obj-m := module.o
```

```
module-objs := file1.o file2.o
```

เป็นการบอกว่าสร้างมอดูลจากไฟล์ module.ko นั้นสร้างจากออปเจ็กไฟล์ file1.o และ file2.o

เมกไฟล์ด้านบนที่กล่าวมายังไม่สามารถใช้งานได้ เพราะยังขาดในส่วนofdตำแหน่งของซอร์สโค้ดของเคอร์เนล ซึ่งจำเป็นต้องใช้ในการคอมไพล์ด้วยเช่นถ้าเคอร์เนลซอร์สของคุณอยู่ที่ตำแหน่ง ~/kernel-2.6 เช่นตัวอย่างนี้

```
make -C ~/kernel-2.6 M=`pwd` modules
```

คำสั่งจะเริ่มจากการเปลี่ยนไดเรกทอรีไปเป็นไดเรกทอรีที่ระบุไว้ด้วยออปชัน -C (ซึ่งระบุตำแหน่งของเคอร์เนลซอร์ส) ออปชัน M= ทำให้เมกไฟล์ย้ายกลับมาที่ซอร์สไดเรกทอรีของมอดูลก่อนที่จะบิลด์ modules ซึ่ง modules นั้นถูกระบุไว้ในตัวแปร obj-m หลังจากนั้นเราก็ใช้คำสั่ง make ที่คอมมานไลน์ โปรแกรม make จะไปอ่านเมกไฟล์แล้วทำงานเอง

4.4 คาเร็กเตอร์ดีไวซ์ไดรเวอร์

การเขียนโปรแกรมไดรเวอร์ของคาเร็กเตอร์ดีไวซ์ เป็นแบบที่ง่ายที่สุด เนื่องจากการไม่มีการเกี่ยวข้องกับบัฟเฟอร์ โดยไดรเวอร์จะทำงานในลักษณะของมอดูล คือ สามารถโหลดได้เมื่อเคอร์เนลต้องการใช้งาน ซึ่งทำงานเหมือนเป็นอุปกรณ์จริง

เคอร์เนลจะใช้เลขเมเจอร์เพื่อกำหนดไดรเวอร์ที่จะใช้กับอุปกรณ์ และดีไวซ์ไดรเวอร์จะใช้เลขไมเนอร์ในการแยกแยะอุปกรณ์แต่ละชิ้น ซึ่งการจะเข้าถึงเลขเมเจอร์และเลขไมเนอร์ได้นั้น จะใช้ตัวแปรที่สร้างจาก dev_t (ประกาศไว้ใน linux/types.h) วิธีการเซตค่าเมเจอร์และไมเนอร์นั้นกระทำผ่านมาโครที่ประกาศไว้ในไฟล์ <linux/kdev_t.h> โดยสามารถแบ่งได้เป็น

```
MAJOR(dev_t dev);
```

```
MINOR(dev_t dev);
```

ถ้าคุณมีเลขเมเจอร์และไมเนอร์แล้วต้องการใส่ลงใน dev_t ให้ใช้

```
MKDEV(int major, int minor);
```

4.5 การจองและคืนเลขดีไวซ์

สิ่งแรกที่ไดรเวอร์จำเป็นต้องทำในการติดตั้งคาเร็กเตอร์ดีไวซ์คือให้ได้รับจำนวนดีไวซ์หนึ่งหรือมากกว่าที่จะทำงาน มีฟังก์ชันที่สำคัญคือ register_chrdev_region ซึ่งได้ประกาศไว้ใน <linux/fs.h>

```
int register_chrdev_region(dev_t first, unsigned int count, char *name);
```

โดยที่ first เลขเริ่มต้นที่อยู่ในช่วงที่คุณต้องการจอง โดยมากจะเป็นเลข 0 count คือจำนวนของเลขของดีไวซ์ที่ต่อเนื่องกันทั้งหมด สุดท้าย name เป็นชื่อของดีไวซ์ที่จะไปปรากฏใน /proc/devices และ sysfs

บ่อยครั้งที่ไม่ว่าเลขเมเจอร์ที่ดีไวซ์จะใช้ ก็สามารถใช้การวิธีการจอง ณ เวลาที่ติดตั้งนั้นเลย (dynamicly-allocated) ได้ โดยเคอร์เนลจะจองเลขเมเจอร์ให้เลย ณ ตอนที่ติดตั้ง แต่คุณต้องใช้ฟังก์ชัน

```
int alloc_chrdev_region(dev_t *dev, unsigned int firstminor,
                        unsigned int count, char *name);
```

ฟังก์ชันนี้ dev จะเป็นเอาต์พุตเดียวที่ได้รับถ้า ฟังก์ชันนั้นทำสำเร็จ firstminor จะเป็นเลขไมเนอร์ตัวแรกที่จะใช้ มักจะเป็น 0count และ name นั้นเหมือนกับฟังก์ชัน register_chrdev_region ส่วนตอนการคืนเลขดีไวซ์ใช้ฟังก์ชัน

```
void unregister_chrdev_region(dev_t first, unsigned int count);
```

ซึ่งจะถูกเรียกใช้งานในฟังก์ชันที่กำจัดมอดูล

4.6 ฟังก์ชันที่ใช้ทำงานกับอุปกรณ์ (ไฟล์)

เคอร์เนลจะทำงานกับอุปกรณ์ผ่านรูทีนของไดรเวอร์ที่เก็บไว้ในโครงสร้างข้อมูล file_operations โดยในกรณีที่ฟังก์ชันใดไม่ได้ทำงานกับอุปกรณ์ จะไม่มีการใส่ค่าในพอยเตอร์นั้น โดยโครงสร้างของ file_operations มีดังนี้

```
struct file_operations fops = {
    .owner =    THIS_MODULE,
    .llseek =   llseek,
    .read =     read,
    .write =    write,
    .ioctl =    ioctl,
    .open =     open,
    .release =  release,
};
```

ในโครงสร้างของ file_operations นั้นจะประกอบด้วยพอยเตอร์ไปยังฟังก์ชันต่าง ๆ เช่น .read ก็จะชี้ไปยังฟังก์ชันที่อยู่หลังเครื่องหมายเท่ากับในที่นี้ก็คือฟังก์ชัน read

สรุปการทำงานของฟังก์ชัน ดังต่อไปนี้

```
struct module *owner
```

เป็นฟิลด์แรกใน file_operations ซึ่งไม่ได้เป็นโอเปอเรชันทั้งหมด แต่เป็นพอยเตอร์ที่บอก ว่าถึงเจ้าของมอดูลนี้ ฟิลด์นี้จะช่วยป้องกันมอดูลจากการถูกเอาออก (unload) ขณะที่โอเปอเรชัน ยังคงใช้งานอยู่ โดยปกติแล้วจะถูกเซตไว้เป็น THIS_MODULE มาโครนี้จะถูกกำหนดไว้ใน <linux/module.h>

```
loff_t (*llseek) (struct file *, loff_t, int);
```

ใช้เปลี่ยนตำแหน่งในการอ่านหรือเขียนข้อมูลลงไฟล์ โดยค่าที่ส่งกลับจะเป็นจำนวนเต็ม บวก หรือจุดออฟเซต (Offset) จากจุดเริ่มต้นไฟล์ ถ้าไม่มีการระบุฟังก์ชันนี้ จะทำให้ใครเวอร์ไม่ สามารถกำหนดตำแหน่งบนข้อมูลของอุปกรณ์ได้

```
ssize_t (*read) (struct file *, char __user *, size_t, loff_t *);
```

ใช้เอ็กเซตข้อมูลจากอุปกรณ์ ถ้ามีการกำหนดฟังก์ชันนี้เป็น NULL จะทำให้ฟังก์ชันของ ระบบ read ไม่สามารถทำงานได้ จะเกิด ERROR -EINVAL โดยค่าที่ส่งกลับจะเป็นจำนวนไบต์ที่ อ่านได้ เมื่อทำงานสำเร็จ

```
ssize_t (*write) (struct file *, const char __user *, size_t, loff_t *);
```

ฟังก์ชันจะส่งข้อมูลไปยังอุปกรณ์ โดยค่าที่ส่งกลับจะเป็นจำนวนไบต์ที่เขียนได้

```
int (*readdir) (struct file *, void *, filldir_t);
```

ฟังก์ชันนี้ ควรจะกำหนดให้เป็น NULL สำหรับไฟล์ node ของอุปกรณ์ เนื่องจากปกติจะใช้ อ่านข้อมูลในไดเรกทอรี (Directory)

```
int (*ioctl) (struct inode *, struct file *, unsigned int, unsigned long);
```

ใช้สำหรับการออกคำสั่งเฉพาะของอุปกรณ์ เช่น การฟอร์แมต (Format) ดิสก์ โดยไม่มีการอ้างอิงไปยังตาราง fops โดยค่าที่ส่งกลับจะเป็นจำนวนเต็มบวก

```
int (*mmap) (struct file *, struct vm_area_struct *);
```

ใช้แมป (Map) พื้นที่หน่วยความจำของอุปกรณ์และหน่วยความจำของโปรเซส

```
int (*open) (struct inode *, struct file *);
```

ใช้เมื่อต้องการจะเริ่มการทำงานกับอุปกรณ์ โดยถ้ามีการกำหนดฟังก์ชันนี้เป็น NULL จะ ทำให้การเรียกใช้งานฟังก์ชันสำเร็จเสมอ แต่ไม่มีค่าส่งกลับ

```
int (*release) (struct inode *, struct file *);
```

ใช้งานเมื่อไฟล์ดีไวซ์ใดเวอร์ (/dev/...) ของอุปกรณ์ถูกปิด

```
int (*fsync) (struct file *, struct dentry *, int);
```

รีเซ็ต (Reset) อุปกรณ์ให้อยู่ในสถานะพร้อมทำงาน